

Washermann

BACKEND API DOCUMENTATION

Notifications System Developer Reference

Complete technical guide covering all five notification channels, every event trigger, admin template management, and direct API testing.



Email (Resend)



SMS (Termii)



Push (FCM)



In-App



WhatsApp (Twilio)

VERSION

4.0.0

BASE URL

<https://api.washermann.com>

AUTH

JWT Bearer

DATE

May 2026

Table of Contents

1. System Overview & Architecture
 - a. Five-channel design
 - b. Environment configuration
 - c. How templates work
2. Channel Reference
 - a. Email (Resend)
 - b. SMS (Termii)
 - c. Push Notifications (Firebase)
 - d. In-App Inbox
 - e. WhatsApp (Twilio)
3. Event & Channel Matrix
4. In-App Inbox API
5. Admin Template Management API
6. FCM Token Registration
7. Direct API Testing (cURL & Postman)
8. Use Cases & Scenario Walkthroughs
9. Design Decisions & Justification
10. Environment Variables Reference

1

System Overview & Architecture

How the notification system is structured and why

The Washermann notification system is a unified, five-channel messaging layer built inside a **NestJS global module**. Every business event that matters to a customer, rep, vendor, or admin triggers the appropriate channel mix automatically — the calling code only calls one method.



Architecture



Fire & Forget

NotificationsModule is decorated

`@Global()`, meaning its

NotificationsService is injectable everywhere without re-importing. Calling code simply injects the service and calls a semantic method like

`notifyOrderPlaced()`.

Every notification call is wrapped in a fire-and-forget pattern. A transient Resend outage or an expired FCM token will **never crash a business transaction**. Failures are logged as warnings only.

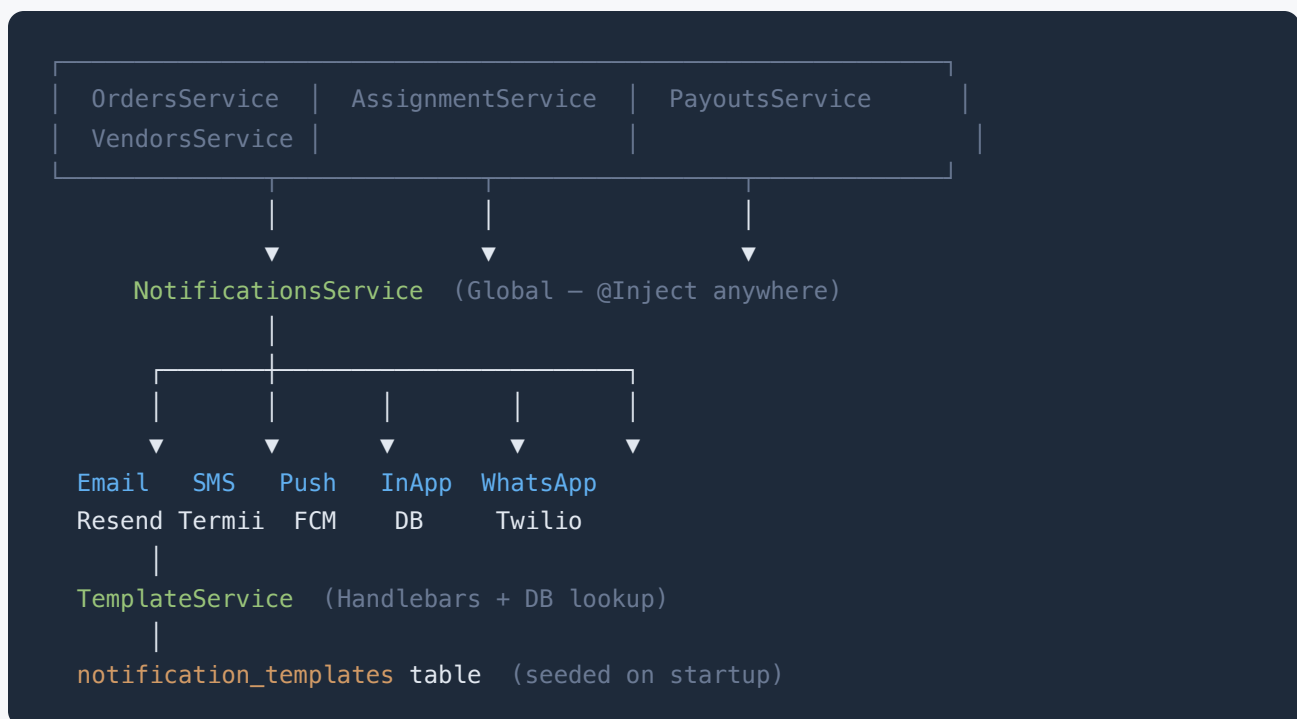
DB-Backed Templates

50+ default templates seed the `notification_templates` table on first startup. Admins can update content, styling, subject lines, and HTML via the REST API without redeployment.

Handlebars Rendering

Template bodies use `{{variableName}}` syntax. The **TemplateService** compiles and renders them at send time with real data. DB templates take precedence over code defaults.

Service Dependency Map



Environment Configuration

Add these keys to your `.env` file to enable each channel:

Variable	Channel	Example
RESEND_API_KEY	EMAIL	re_XXXXXXXXXXXXXXXXXX
RESEND_FROM_EMAIL	EMAIL	no-reply@washermann.com
RESEND_FROM_NAME	EMAIL	Washermann
TERMII_API_KEY	SMS	TLXXXXXXXXXXXXXXXXXX
TERMII_SENDER_ID	SMS	Washermann
FIREBASE_PROJECT_ID	PUSH	washermann-prod
FIREBASE_CLIENT_EMAIL	PUSH	firebase-adminsdk@...
FIREBASE_PRIVATE_KEY	PUSH	-----BEGIN RSA... (use \n for newlines)
TWILIO_ACCOUNT_SID	WHATSAPP	ACXXXXXXXXXXXXXXXXXX
TWILIO_AUTH_TOKEN	WHATSAPP	XXXXXXXXXXXXXXXXXX
TWILIO_WHATSAPP_FROM	WHATSAPP	whatsapp:+14155238886
ADMIN_NOTIFICATION_EMAIL	EMAIL	ops@washermann.com



Graceful degradation: If a channel's credentials are absent, that channel is silently disabled at startup and a warning is logged. All other channels continue to function normally. No **500** errors are thrown.

2

Channel Reference

Provider details, rate limits, and when each channel fires



Email

Resend API ·
HTML templates
· deep detail



SMS

Termii · plain
text · 160 chars ·
high open rate



Push

Firebase FCM ·
iOS & Android ·
device token



In-App

PostgreSQL ·
REST inbox ·
read/unread



WhatsApp

Twilio ·
conversational ·
high
engagement



Email — Resend

Provider: [Resend](#) — modern transactional email API. Free tier: 100 emails/day, 3,000/month. Paid from \$20/month.

Format: Full responsive HTML generated via the `buildEmailHtml()` helper. Admins can override the HTML body entirely via the template API. The email renders a header, styled content block, and footer.

When it fires: Order confirmations, delivery notifications, payout confirmations/failures, vendor verification, pricing approvals — all events where a permanent record or rich formatting is valuable.

Not used for: Real-time alerts (push/WhatsApp are faster). Short operational messages (use SMS).



SMS — Termii

Provider: [Termii](#) — Nigerian-focused SMS gateway with excellent local delivery rates. Sender ID registration required for production.

Format: Plain text, concise (under 160 characters where possible to avoid multi-part billing). No links unless absolutely necessary.

When it fires: Order placed (backup to push for users without the app), order delivered, payout approved, assignment broadcast to rep/vendor, vendor verification. SMS fires when the user has a phone number registered.

Billing note: SMS has per-message cost. Use for high-value events only — the system does not send SMS for every status change.

Push — Firebase Cloud Messaging (FCM)

Provider: [Firebase Admin SDK](#) — supports iOS (APNs bridged), Android, and Web.

Requirements: The mobile app must call `PATCH /users/me/fcm-token` at login and whenever the device token refreshes. Stale tokens (uninstalled app) are handled gracefully — logged as `WARN`, not errors.

When it fires: Every actionable event — order placed, rep/vendor assigned, pickup, delivery, earnings, payouts, assignments. Push is the primary real-time channel for app users.

Priority: Android: `high` (wakes device). iOS APNs: `badge: 1, sound: default`. Channel ID: `washermann_default`.

Multi-cast: `PushService.sendMulti(tokens[], title, body)` supports up to 500 tokens per batch via `sendEachForMulticast`.

In-App Notifications

Storage: PostgreSQL table `in_app_notifications`. Each notification is a row with `userId`, `title`, `body`, `type`, `isRead`, `readAt`, and optional `metadata` JSONB.

Types: `order` | `assignment` | `payout` | `account` | `general`

When it fires: Every event fires an in-app notification (it's free and persistent). The mobile app polls `GET /notifications/me/unread-count` on launch to show the badge count.

Retention: Notifications are never auto-deleted — they form a permanent audit trail visible to the user.

WhatsApp — Twilio

Provider: [Twilio WhatsApp Business API](#). For sandbox testing, use the Twilio sandbox number. For production, you need a WABA (WhatsApp Business Account) approved number.

Format: Rich text with `*bold*` and `_italic_` markdown. Emoji supported. Can include multi-line messages.

When it fires: High-engagement events — order placed, rep assigned, pickup, delivery, assignment broadcasts, payout approved. WhatsApp has ~98% open rates in Nigeria.

Phone format: Numbers must be in E.164 format (e.g. `+2348012345678`). The system auto-prefixes `whatsapp:` for Twilio.



Event & Channel Matrix

Which channels fire for each platform event

The table below shows every event the notification system handles and which channels are used.  = fires for this channel, — = not used.

Event	Audience	 Email	 SMS	 Push	 In-App	 WA
📦 Order Events						
Order Placed	Customer	✓	✓	✓	✓	✓
Rep Assigned	Customer	—	—	✓	✓	✓
Laundry Picked Up	Customer	—	—	✓	✓	✓
Laundry Picked Up	Vendor	—	—	✓	✓	✓
Order Delivered	Customer	✓	✓	✓	✓	✓
Order Completed (Customer)	Customer	✓	—	✓	✓	—
Earning Credited	Vendor	✓	—	✓	✓	—
Earning Credited	Rep	—	—	✓	✓	—
Order Cancelled	Customer	✓	✓	✓	✓	—
🚚 Assignment Events						
Assignment Broadcast	Rep	—	✓	✓	✓	✓
Assignment Confirmed	Rep	—	—	✓	✓	—
Assignment Broadcast	Vendor	—	✓	✓	✓	✓
No Reps Available (Escalation)	Admin	✓	—	—	—	—
🏢 Vendor Events						
Vendor Account Verified	Vendor	✓	✓	✓	✓	—
Vendor Pending Verification	Admin	✓	—	—	—	—
Pricing Approved	Vendor	✓	—	✓	✓	—
🚲 Rep Events						
Rep Flagged for Review	Rep	✓	—	✓	✓	—
💰 Payout Events						

New Payout Request	Admin		—	—	—	—
Payout Approved	Vendor					
Payout Failed	Vendor					—

4

In-App Inbox API

REST endpoints for the user notification inbox

These endpoints power the notification bell / inbox feature in the mobile and web app. All require a valid JWT Bearer token.

Returns the authenticated user's in-app notification inbox, newest first. Paginated.

Query Parameters

Parameter	Type	Default	Description
page	integer	1	Page number (1-based)
limit	integer	20	Items per page (max 50)

Response

```
{
  "data": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "userId": "d7e3b8c2-...",
      "title": "Order Confirmed",
      "body": "Your order WM-20240501-001 has been placed (1200 WP).",
      "type": "order",
      "isRead": false,
      "readAt": null,
      "metadata": { "orderRef": "WM-20240501-001" },
      "createdAt": "2024-05-01T10:30:00Z"
    }
  ],
  "meta": {
    "total": 42,
    "page": 1,
    "limit": 20,
    "pages": 3
  }
}
```

GET /notifications/me/unread-count

USER

Returns the count of unread notifications for the current user. Intended for badge indicators — call this on app launch and after marking reads.

Response

```
{ "count": 7 }
```

POST /notifications/me/:id/read

USER

Marks a single notification as read. Idempotent — calling on an already-read notification is safe.

Path Parameters

Parameter	Type	Description
<code>id</code>	UUID	The notification ID to mark read

Response

```
{
  "id": "550e8400-...",
  "isRead": true,
  "readAt": "2024-05-01T11:00:00Z"
}
```

POST /notifications/me/read-all

USER

Marks all of the current user's unread notifications as read in a single batch update. Ideal for a "Mark all as read" button.

Response

```
{ "updated": 7 }
```

5

Admin Template Management API

Manage notification content, styling, and branding without redeployment

All endpoints under `/admin/notification-templates` require **Admin role**. Templates are keyed by `(key, channel)` pairs — e.g. `order.placed.customer` + `email`.

Template Object Structure

```
{
  "id": "uuid",
  "key": "order.placed.customer", // event identifier
  "channel": "email", // email | sms | push | in_app | whatsapp
  "name": "Order Placed – Customer Email",
  "subject": "Order Confirmed: {{orderRef}} 🍷",
  "body": "Hi {{customerName}}, your order {{orderRef}} has been placed...",
  "htmlBody": "<!DOCTYPE html>...", // full email HTML (nullable)
  "emailStyle": { // JSONB – email only
    "primaryColor": "#1a1a2e",
    "accentColor": "#4fc3f7",
    "bodyBgColor": "#f5f5f5",
    "cardBgColor": "#ffffff",
    "textColor": "#444444",
    "logoUrl": "https://cdn.washermann.com/logo.png",
    "logoAlt": "Washermann",
    "footerText": "© {{year}} Washermann. All rights reserved.",
    "fontFamily": "-apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif"
  },
  "variables": ["customerName", "orderRef", "totalWP"],
  "isActive": true,
  "updatedBy": "admin-uuid",
  "createdAt": "2024-01-01T00:00:00Z",
  "updatedAt": "2024-05-01T10:00:00Z"
}
```

GET /admin/notification-templates

ADMIN

List all notification templates. Filter by channel or event key.

Query Parameters

Parameter	Type	Description
<code>channel</code>	string	<code>email</code> <code>sms</code> <code>push</code> <code>in_app</code> <code>whatsapp</code>
<code>key</code>	string	Exact event key, e.g. <code>order.placed.customer</code>

GET /admin/notification-templates/:id

ADMIN

Retrieve a single template by UUID including full HTML body and email style.

Update any field on a template. All fields are optional — only include what you want to change.

Request Body

Field	Type	Description
<code>name</code>	string	Human-readable template name
<code>subject</code>	string	Email subject line (Handlebars variables supported)
<code>body</code>	string	Plain-text body (used for SMS, push, in-app, WhatsApp)
<code>htmlBody</code>	string	Full HTML email body. Set to <code>null</code> to revert to auto-generated layout
<code>emailStyle</code>	object	Partial style overrides (merged with existing). See EmailStyle schema above
<code>isActive</code>	boolean	Set <code>false</code> to disable without deleting (falls back to hardcoded default)
<code>variables</code>	string[]	List of available Handlebars variables (informational for docs)

Example — Update brand colours

```
{
  "emailStyle": {
    "primaryColor": "#0d47a1",
    "accentColor": "#ffab40",
    "logoUrl":      "https://cdn.washermann.com/logo-v2.png",
    "footerText":   "© {{year}} Washermann Ltd. Lagos, Nigeria."
  }
}
```


POST /admin/notification-templates/:id/preview

ADMIN

Renders the template with placeholder values (variable names shown as `{{variableName}}`). Returns `subject` , `body` , and `htmlBody` . Use this to verify email rendering before going live.

Request Body (optional)

```
{
  "emailStyle": {
    "primaryColor": "#0d47a1"
  }
}
```

Response

```
{
  "subject": "Order Confirmed: {{orderRef}} 🛒",
  "body": "Hi {{customerName}}, your order {{orderRef}} has been placed...",
  "htmlBody": "<!DOCTYPE html><html>...full rendered HTML...</html>"
}
```

POST /admin/notification-templates/:id/reset

ADMIN

Resets the template to the original hardcoded defaults. Clears all admin customisations. Non-destructive — the UUID remains the same.

All Event Keys Reference

Key	Audience	Available Channels
<code>order.placed.customer</code>	Customer	email, sms, push, in_app, whatsapp
<code>order.rep_assigned.customer</code>	Customer	push, in_app, whatsapp
<code>order.picked_up.customer</code>	Customer	push, in_app, whatsapp
<code>order.picked_up.vendor</code>	Vendor	push, in_app, whatsapp
<code>order.delivered.customer</code>	Customer	email, sms, push, in_app, whatsapp
<code>order.completed.customer</code>	Customer	email, push, in_app
<code>order.cancelled.customer</code>	Customer	email, sms, push, in_app
<code>order.earning_credited.vendor</code>	Vendor	email, push, in_app
<code>order.earning_credited.rep</code>	Rep	push, in_app
<code>assignment.broadcast.rep</code>	Rep	sms, push, in_app, whatsapp
<code>assignment.confirmed.rep</code>	Rep	push, in_app
<code>assignment.broadcast.vendor</code>	Vendor	sms, push, in_app, whatsapp
<code>assignment.no_reps.admin</code>	Admin	email
<code>vendor.account_verified</code>	Vendor	email, sms, push, in_app
<code>vendor.pending_verification.admin</code>	Admin	email
<code>pricing.approved.vendor</code>	Vendor	email, push, in_app
<code>rep.flagged_for_review</code>	Rep	email, push, in_app
<code>payout.new_request.admin</code>	Admin	email
<code>payout.approved.vendor</code>	Vendor	email, sms, push, in_app, whatsapp
<code>payout.failed.vendor</code>	Vendor	email, sms, push, in_app

6

FCM Token Registration

Register and refresh device tokens for push notifications

Registers or updates the authenticated user's FCM device token. Call this at login and whenever `onTokenRefresh` fires in the mobile app.

Request Body

```
{ "token": "fcm_device_token_from_firebase_sdk" }
```

Response

```
{ "message": "FCM token updated" }
```



Pass `"token": ""` (empty string) or `"token": null` to clear the token (e.g. on logout). This prevents push delivery to a signed-out device.

7

Direct API Testing

cURL examples for every notification-related endpoint

Replace `{{BASE_URL}}` with your server address (e.g. `http://localhost:3000`) and `{{TOKEN}}` with a valid JWT from a login response.

Step 0 — Get a Token

```
# Login as customer
curl -s -X POST {{BASE_URL}}/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"customer@test.com","password":"Test1234!"}' \
  | jq '.access_token'

# Save to shell variable
export TOKEN="eyJhbGciOiJ..."
export BASE_URL="http://localhost:3000"
```

Step 1 — Register FCM Token (Mobile App Required)

```
curl -X PATCH ${BASE_URL}/users/me/fcm-token \
  -H "Authorization: Bearer ${TOKEN}" \
  -H 'Content-Type: application/json' \
  -d '{"token":"dJHGk1...FCMTokenFromFirebase}"'
```

Step 2 — Trigger a Notification (Place an Order)

Placing an order fires: email + SMS + push + in-app + WhatsApp to the customer simultaneously.

```
curl -X POST ${BASE_URL}/orders \
  -H "Authorization: Bearer ${TOKEN}" \
  -H 'Content-Type: application/json' \
  -d '{
    "areaId":      "area-uuid-here",
    "serviceType": "wash_and_fold",
    "bagSize":     "medium",
    "pickupAddress": "15 Admiralty Way, Lekki Phase 1",
    "scheduledPickupAt": "2024-05-10T09:00:00Z"
  }'

# Expected: 201 Created
# Side effects: 5 notification channels fire in background (fire-and-forget)
```

Step 3 — Check the In-App Inbox

```
# Get all notifications (page 1)
curl -s ${BASE_URL}/notifications/me \
  -H "Authorization: Bearer ${TOKEN}" | jq '.data[0]'

# Get unread count (for badge)
curl -s ${BASE_URL}/notifications/me/unread-count \
  -H "Authorization: Bearer ${TOKEN}"
# → {"count": 1}
```

Step 4 — Mark Notifications Read

```
# Mark single notification read
export NOTIF_ID="550e8400-e29b-41d4-a716-446655440000"
curl -X POST ${BASE_URL}/notifications/me/${NOTIF_ID}/read \
  -H "Authorization: Bearer ${TOKEN}"

# Mark ALL read
curl -X POST ${BASE_URL}/notifications/me/read-all \
  -H "Authorization: Bearer ${TOKEN}"
# → {"updated": 1}
```

Step 5 — Admin: List Notification Templates

```
# Get admin token first
export ADMIN_TOKEN="eyJhbGciOi..."

# List all templates
curl -s ${BASE_URL}/admin/notification-templates \
  -H "Authorization: Bearer ${ADMIN_TOKEN}" | jq '.[0]'

# Filter by channel
curl -s "${BASE_URL}/admin/notification-templates?channel=email" \
  -H "Authorization: Bearer ${ADMIN_TOKEN}" | jq 'length'

# Filter by event key
curl -s "${BASE_URL}/admin/notification-templates?key=order.placed.customer" \
  -H "Authorization: Bearer ${ADMIN_TOKEN}"
```

Step 6 — Admin: Update a Template's Brand Colours

```
export TEMPLATE_ID="uuid-of-template"

curl -X PATCH ${BASE_URL}/admin/notification-templates/${TEMPLATE_ID} \
  -H "Authorization: Bearer ${ADMIN_TOKEN}" \
  -H 'Content-Type: application/json' \
  -d '{
    "emailStyle": {
      "primaryColor": "#0d47a1",
      "accentColor": "#ffab40",
      "logoUrl": "https://cdn.washermann.com/logo-v2.png",
      "footerText": "© {{year}} Washermann Ltd. All rights reserved."
    }
  }'
```

Step 7 — Admin: Update SMS Body

```
# Find the SMS template for order.placed.customer
curl -s "${BASE_URL}/admin/notification-templates?key=order.placed.customer&channel=sms" \
  -H "Authorization: Bearer ${ADMIN_TOKEN}" | jq '.[0].id'

# Update the body
curl -X PATCH ${BASE_URL}/admin/notification-templates/${TEMPLATE_ID} \
  -H "Authorization: Bearer ${ADMIN_TOKEN}" \
  -H 'Content-Type: application/json' \
  -d '{
    "body": "Hi {{customerName}}! 🟢 Order {{orderRef}} confirmed. {{totalWP}} WP charge"
  }'
```

Step 8 — Admin: Preview Template Rendering

```
curl -X POST ${BASE_URL}/admin/notification-templates/${TEMPLATE_ID}/preview \
  -H "Authorization: Bearer ${ADMIN_TOKEN}" \
  -H 'Content-Type: application/json' \
  -d '{
    "emailStyle": { "primaryColor": "#0d47a1" }
  }'

# Returns: { subject, body, htmlBody }
# Copy htmlBody into a browser to inspect email rendering
```

Step 9 — Admin: Reset Template to Default

```
curl -X POST ${BASE_URL}/admin/notification-templates/${TEMPLATE_ID}/reset \
  -H "Authorization: Bearer ${ADMIN_TOKEN}"

# Reverts all content to hardcoded defaults
```

Postman Quick Setup

Import the collection at `/postman/washermann.postman_collection.json`. Set the following collection variables:

Variable	Description
<code>baseUrl</code>	<code>http://localhost:3000</code>
<code>authToken</code>	JWT from POST /auth/login
<code>adminToken</code>	JWT from POST /auth/login (admin user)

The **In-App Notifications** and **Admin Notification Templates** folders contain all requests pre-configured.

8

Use Cases & Scenario Walkthroughs

Real-world scenarios showing how notifications flow end-to-end

Scenario A: Complete Order Lifecycle

Shows every notification a customer receives from order placement to completion.

Customer: Chioma places a laundry order

Step	Trigger	Channels	Message Preview
1	Order placed (POST /orders)	EMAIL PUSH WA SMS IN-APP	"Order WM-001 confirmed. 1200 WP charged."
2	Rep accepts assignment	PUSH WA IN-APP	"Emeka has been assigned to your order."
3	Rep marks order PICKED_UP	PUSH WA IN-APP	"Your laundry is on its way to the vendor."
4	Rep marks order DELIVERED	EMAIL PUSH WA SMS IN-APP	"Your laundry has been delivered! Please confirm."
5	Customer confirms / 24h auto-complete	EMAIL PUSH IN-APP	"Order WM-001 complete. Thank you for using Washermann!"

Scenario B: Rep Assignment Broadcast

Shows what happens when the system broadcasts a job to reps.

Reps: Batch broadcast → first accepts wins

1. **System** scores available reps in the area → picks top 3.

2. **All 3 reps** receive: SMS + Push + In-App + WhatsApp simultaneously. "New job available! Order WM-001 — 15 Admiralty Way. Tap to accept!"
3. **First rep to accept** (within 90-second window) gets the job. System cancels all other pending broadcasts.
4. **Winning rep** receives Push + In-App: "Job confirmed  — head to pickup."
5. **Customer** receives Push + In-App + WhatsApp: "Emeka has been assigned to your order."
6. If nobody accepts → system re-broadcasts to next batch. If all batches exhausted → admin email alert.

Scenario C: Vendor Payout Flow

Vendor requests a payout — admin approves

Actor	Action	Notification Sent To
Vendor	Submits payout request (POST /payouts)	 → Admin: "Folake requested ₦45,000 payout."
Admin	Approves (POST /admin/payouts/:id/approve)	Vendor gets: Email + SMS + Push + In-App + WhatsApp: "₦45,000 payout approved. Bank transfer initiated."
Paystack	Transfer fails (bank issue)	Vendor gets: Email + SMS + Push + In-App: "Payout of ₦45,000 failed. Balance unchanged. Contact support."

Scenario D: Vendor Onboarding

New vendor submits documents

1. **Admin team** receives email: "New vendor 'Clean Crisp Laundry' awaiting verification. Vendor ID: xxx."
2. **Admin verifies** → **Vendor** receives: Email + SMS + Push + In-App — "Congratulations! Your account is verified. Turn on availability to start receiving orders."

3. **Vendor submits pricing** → Admin reviews → **Vendor** receives: Email + Push + In-App — "Your pricing has been approved. New rates are live."

Scenario E: Changing Brand Colours (No Redeployment)

Marketing wants to update email colours for a campaign

1. Admin calls `GET /admin/notification-templates?channel=email` → gets all 10 email template IDs.
2. Admin calls `PATCH /admin/notification-templates/:id` for each with `emailStyle: { primaryColor: "#e91e63", accentColor: "#fff176" }`.
3. Admin uses `POST /admin/notification-templates/:id/preview` to verify the HTML rendering before committing.
4. New colours take effect immediately for all subsequent emails — no code change, no restart.
5. After the campaign, `POST /admin/notification-templates/:id/reset` reverts to defaults.



Design Decisions & Justification

Why the system is built this way

1. Why five channels instead of one or two?

Different channels serve different functions and user contexts:

Channel	Strength	Limitation
Email	Rich formatting, permanent record, works offline	Not seen immediately; low urgency
SMS	Works on feature phones; high open rate even without data	Per-message cost; 160-char limit
Push	Instant; app badge; works when app is closed	Requires app install + token
In-App	Always available; searchable history; no cost	Only seen when app is open/refreshed
WhatsApp	98%+ open rate in Nigeria; conversational context	Requires Twilio WABA approval

A customer without the app still gets SMS + WhatsApp. A customer with the app gets Push + In-App for instant feedback. Email provides a paper trail for financial events. This redundancy ensures no critical notification is missed regardless of device or connectivity state.

2. Why fire-and-forget?

Notification delivery is non-critical to the transaction outcome. A failed email must never roll back an order or payout. The `fire(fn, logger, label)` wrapper ensures:

- The business operation returns immediately (low latency for the user)
- Notification errors are logged for observability but suppressed from the HTTP response
- Temporary provider outages (Resend/Termii/FCM/Twilio) don't cause user-facing 500 errors

In a future production hardening, failed notifications would be written to a retry queue (Redis/BullMQ) for at-least-once delivery. The current implementation is appropriate for initial launch.

3. Why DB-backed templates with code fallbacks?

Hardcoding templates in source files means every copy change requires a code review, build, and deployment — an unnecessary engineering bottleneck for marketing/operations teams. The DB-backed system allows:

- Non-engineers to update copy, subject lines, and colours via the admin API (or future UI)
- A/B testing of different message variants
- Seasonal branding changes without touching code
- Per-template enable/disable without deployment

The hardcoded `DEFAULT_TEMPLATES` array serves as the safety net — if a DB record is corrupted or missing, the system falls back to working defaults and continues operating.

4. Why Handlebars for variable substitution?

Handlebars was chosen over simple string interpolation because:

- **Familiar syntax** — `{{variableName}}` is readable by non-developers who edit templates in the admin UI
- **Safe compilation** — malformed templates don't crash the process; the service falls back to the raw string
- **Extensible** — Handlebars supports helpers for date formatting, conditionals, loops if needed in future
- **No eval** — unlike template literal tags, Handlebars does not expose arbitrary code execution

5. Why is the module `@Global()`?

NotificationsService needs to be called from `OrdersService`, `AssignmentService`, `PayoutsService`, and `VendorsService` — which are in completely separate modules. Without `@Global()`, every module would need to explicitly import `NotificationsModule`,

adding boilerplate and potential for circular import issues. Making it global is the NestJS-idiomatic pattern for cross-cutting concerns like logging, notifications, and analytics.

6. Why seed templates on startup?

The `TemplateService.seedDefaults()` method runs on `OnModuleInit`. It only inserts rows that don't already exist (idempotent). This means:

- A fresh environment gets all 50+ templates automatically — zero manual setup
- New templates added in a code update are automatically provisioned on next deploy
- Existing admin customisations are never overwritten (the check is `findOne() → skip if exists`)
- Safe to run on every server restart

7. Channel selection logic per event

Not all channels fire for every event — the selection is deliberate:

- **High-value financial events** (payout approved, order placed) → all 5 channels. These are the most important touchpoints and warrant maximum reach.
- **Real-time status updates** (rep assigned, picked up) → push + in-app + WhatsApp. These are transient state changes; email would be noise.
- **Internal admin alerts** (no reps, new payout request) → email only. Admin checks email; push/WhatsApp to a generic admin address isn't meaningful without a dedicated device.
- **Rep earnings** → push + in-app only. Reps use the app; SMS costs money and isn't needed for every earning event.

Environment Variables Reference

Complete .env configuration for all notification channels

```
# — Email (Resend) —————
RESEND_API_KEY=re_your_api_key_here
RESEND_FROM_EMAIL=no-reply@washermann.com
RESEND_FROM_NAME=Washermann

# — SMS (Termii) —————
TERMII_API_KEY=TLyour_api_key_here
TERMII_SENDER_ID=Washermann # Must be approved in Termii dashboard

# — Firebase FCM (Push Notifications) —————
FIREBASE_PROJECT_ID=washermann-prod
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-xxxx@washermann-prod.iam.gserviceaccount.com
FIREBASE_PRIVATE_KEY=-----BEGIN RSA PRIVATE KEY-----\nMIIEo...\n-----END RSA PRIVATE KEY-----
# ↑ Use literal \n for newlines in .env; the config restores real newlines

# — WhatsApp (Twilio) —————
TWILIO_ACCOUNT_SID=ACyour_account_sid_here
TWILIO_AUTH_TOKEN=your_auth_token_here
TWILIO_WHATSAPP_FROM=whatsapp:+14155238886 # Sandbox or approved number

# — Admin notifications —————
ADMIN_NOTIFICATION_EMAIL=ops@washermann.com # Receives payout/escalation alerts

# — Deep link (used in invite emails) —————
DEEP_LINK_BASE=washermann://app
```

Obtaining Credentials

Service	Where to Get	Notes
Resend	resend.com/api-keys	Verify your sending domain in the Resend dashboard for best deliverability.
Termii	termii.com → Dashboard → API	Register your Sender ID. DND (Do Not Disturb) lists are automatically respected.
Firebase	Firebase Console → Project Settings → Service Accounts → Generate Key	Download the JSON file; extract <code>project_id</code> , <code>client_email</code> , <code>private_key</code> .
Twilio WhatsApp	console.twilio.com → Messaging → WhatsApp	Use sandbox for testing. Production requires WABA approval from Meta (1-2 weeks).



Testing checklist:

1. Verify all env vars are set in your `.env` file
2. Start the server and check logs for "initialised" messages from each provider
3. Register FCM token via `PATCH /users/me/fcm-token`
4. Place a test order and check all 5 channels within 10 seconds
5. Verify in-app inbox via `GET /notifications/me`
6. Test admin template update → preview → verify email changed